

Collaborative Whiteboard — Design Doc

Oct 7, 2026 · Ron-ron Aspe Rivera · v2 (Supabase-only, guest-first)

Overview

The whiteboard is a browser app where several people draw on the same infinite canvas at once and see each other's strokes and cursors in under 100 ms. Anyone can start drawing instantly as a guest; signing in with Google or GitHub is optional and keeps boards in a personal dashboard.

The app has **no custom backend server**. The React frontend talks directly to Supabase, which provides auth, the Postgres database, real-time messaging, presence and file storage. The only thing deployed is the static frontend.

Problem it solves. Teams and study groups need a quick shared space to sketch ideas, diagrams and wireframes without installing anything or creating an account first.

Goals

- Real-time drawing for 2–20 users per board with live cursors and presence
- Zero friction: open the site, enter a name, start drawing
- Reliable persistence: nothing drawn is lost on refresh or disconnect
- Simple sharing: invite by link with editor or viewer access
- Deployed publicly and usable from a phone or laptop

Non-goals (v1)

- Offline editing and full CRDT merging
- Video or voice calls
- A custom Node.js backend (saved for a later project)

Portfolio angle. The project shows skills most portfolio CRUD apps don't: real-time event design, optimistic updates with conflict handling, database-enforced security (RLS and Postgres functions), anonymous-to-permanent account upgrades, and canvas rendering performance.

Features

The MVP covers drawing, live collaboration, saving, sharing and guest access; everything else is a stretch goal added after deployment.

Area	MVP	Stretch
Drawing tools	Pen, line, rectangle, ellipse, arrow, text, eraser	Sticky notes, image upload, laser pointer
Styling	Stroke color, stroke width, fill	Opacity, dashed lines, font choice
Canvas	Infinite canvas, pan, zoom, select, move, delete	Resize/rotate handles, snap to grid, layers
Collaboration	Live strokes, live cursors with names, online user list	Follow a user's view, emoji reactions, board chat

Area	MVP	Stretch
History	Per-user undo/redo	Version snapshots and restore
Boards	Create, rename, delete, dashboard with thumbnails	Folders, search, duplicate board
Sharing	Invite link, editor/viewer roles	Public read-only boards, member management UI
Export	Export board as PNG	Export SVG/PDF
Auth	Guest mode (name only), Google and GitHub sign-in, upgrade guest to account	Email magic link, CAPTCHA on guest sign-up

Key user stories

1. As a visitor, I click "Start a board," type my name, and I'm drawing within seconds.
2. As a user, I share a link so a teammate joins in one click, without signing up.
3. As an editor, I draw a shape and my teammate sees it appear instantly.
4. As a collaborator, I see who is online and where their cursor is.
5. As a viewer, I can watch and pan the board but cannot change it.
6. As a guest, I sign in with Google later and keep every board I made as a guest.
7. As a user, I close the tab, come back tomorrow, and the board is exactly as I left it.
8. As a user, I undo my own last action without undoing someone else's.

Tech stack

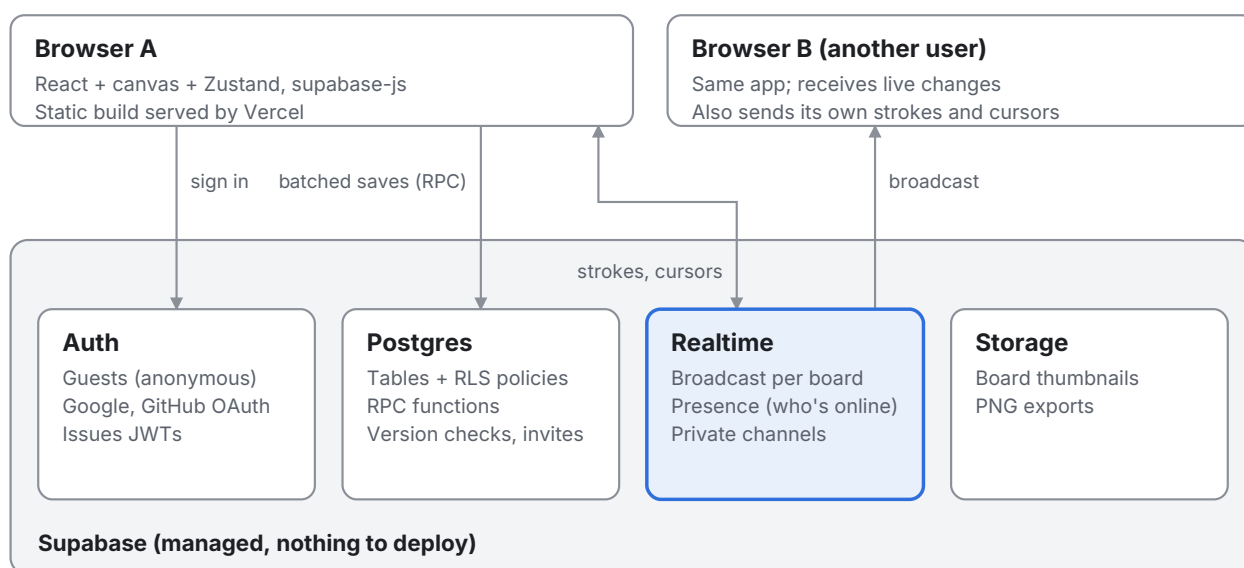
Everything runs on the frontend plus Supabase; there is no server to deploy or keep awake.

Layer	Choice	Why
Frontend	React + Vite + Tailwind CSS	Fast dev server, small bundle, your existing stack
Canvas	Native HTML5 Canvas 2D API	Full control over rendering performance; shows real skill vs. a drawing library
Client state	Zustand	Tiny store that canvas code can read outside React renders
Supabase client	<code>@supabase/supabase-js</code>	One SDK for auth, queries, RPC, realtime and storage
Auth	Supabase Auth	Anonymous sign-in for guests; Google and GitHub OAuth; JWT on every request
Database	Supabase Postgres	Relational data + JSONB elements; RLS enforces permissions
Server logic	Postgres functions (RPC)	Version checks, invite tokens, batched writes run inside the database
Real-time	Supabase Realtime Broadcast	Low-latency messages per board channel (strokes, cursors)
Presence	Supabase Realtime Presence	Built-in "who's online" with automatic cleanup on disconnect
File storage	Supabase Storage	Board thumbnails and exported PNGs

Layer	Choice	Why
Validation	Zod (client) + CHECK constraints (database)	Bad payloads are rejected on both sides
Local dev	Supabase CLI	Runs the full Supabase stack locally; migrations live in the repo
Hosting	Vercel	Static frontend, auto-deploy from GitHub
Testing	Vitest + Playwright	Unit tests for sync logic; end-to-end test with two browser windows

Architecture

The browser talks directly to Supabase over HTTPS and one WebSocket; Postgres is the source of truth, and RLS decides what each user may read or write.



Broadcast carries every change to other users instantly, while each client saves its own changes to Postgres in small batches. Neither path waits for the other, so drawing stays fast even when the database is slow.

Life of one stroke

1. User A draws a rectangle; the client renders it instantly.
2. The client sends `element:upsert` on the Broadcast channel `board:<id>`; User B's canvas draws it.
3. The change is queued, and within 500 ms the client sends the batch to the `apply_element_ops` RPC.
4. Postgres checks A's role and each element's version, saves accepted changes and returns any rejected ones.
5. For a rejected change, the client takes the stored version, redraws it and broadcasts the correction.
6. When anyone opens the board later, it loads from Postgres.

Authentication

Every visitor gets a real Supabase user id, either as a guest or a signed-in user, so permissions work the same way for both.

Mode	How	What they get
Guest	<code>supabase.auth.signInAnonymously()</code> + a display name	Create, join and draw on boards; boards tied to this browser
Google	<code>signInWithOAuth({ provider: 'google' })</code>	Dashboard of boards on any device
GitHub	<code>signInWithOAuth({ provider: 'github' })</code>	Same as Google
Upgrade	Guest links a Google or GitHub identity (<code>linkIdentity</code>)	Keeps the same user id, so every guest board stays theirs

Setup. Create OAuth apps in Google Cloud Console and GitHub Developer Settings, then paste each client ID and secret into the Supabase dashboard under Auth → Providers. Enable anonymous sign-ins and manual identity linking in Auth settings.

Profiles. A database trigger creates a `profiles` row whenever a new auth user appears, copying the display name and avatar. Guests have `is_guest = true` until they link an identity.

Guest cleanup. A scheduled job (`pg_cron`) deletes guest accounts older than 30 days that own no boards, so the auth table doesn't fill with abandoned guests.

Real-time sync design

The board is a set of independent elements, and each change is a small operation on one element, resolved by per-element versioning with last-writer-wins. This avoids CRDT complexity while staying correct for whiteboard use, where two people rarely edit the same shape at the same moment.

Element model. Every stroke, shape or text box is one element: `{ id, type, data, zIndex, version, updatedBy, updatedAt }`. IDs are UUIDs generated on the client, so a shape can be drawn instantly before the database answers.

Channel. Each board uses one private Realtime channel, `board:<id>`. Realtime Authorization policies allow only board members to join it, and only editors and owners to send element events.

Broadcast events

Event	Payload	Saved?	Notes
<code>element:upsert</code>	<code>{ element }</code>	Yes, via RPC	New or changed element
<code>element:delete</code>	<code>{ id, version }</code>	Yes, via RPC	Soft delete (sets <code>deleted_at</code>)
<code>stroke:progress</code>	<code>{ id, points[] }</code>	No	In-progress pen points, throttled to 1 per 30 ms
<code>cursor:move</code>	<code>{ x, y }</code>	No	Throttled to 1 per 50 ms
<code>board:renamed</code>	<code>{ title }</code>	Already saved	Keeps headers in sync

Presence state per user: `{ userId, name, color, isGuest, tool }`. Supabase removes a user from presence automatically when their connection drops.

Conflict handling. Each element carries a `version` integer. The `apply_element_ops` function accepts an operation only if its version equals the stored version plus one, so stale updates are rejected inside the database where no client can skip the check.

Optimistic updates. The client applies its own change immediately and rolls back only if the RPC rejects it.

Undo/redo. Each client keeps its own stack of inverse operations. Undo sends a normal upsert or delete, so it never touches other users' work.

Joining without gaps

1. Subscribe to the channel and start buffering incoming events.
2. Fetch all live elements for the board from Postgres.
3. Apply buffered events whose version is newer than the fetched one, then stop buffering.

Reconnects. After a dropped connection, the client resubscribes, refetches elements changed since its last sync (`updated_at > lastSync`) and resends any operations the RPC never confirmed.

Database design

Four tables hold everything: `profiles`, `boards`, `board_members` and `elements`. Shape-specific fields live in a JSONB column, so new tools need no migration.

```
-- enums
create type board_visibility as enum ('private', 'link', 'public');
create type member_role      as enum ('owner', 'editor', 'viewer');
create type element_type     as enum ('pen', 'line', 'rect', 'ellipse', 'arrow', 'text', 'sticky',
'image');

-- 1 row per user, created by a trigger on auth.users
create table profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  display_name text not null check (char_length(display_name) between 1 and 40),
  avatar_url  text,
  is_guest    boolean not null default true,
  created_at  timestampz not null default now()
);

create table boards (
  id          uuid primary key default gen_random_uuid(),
  owner_id    uuid not null references profiles(id) on delete cascade,
  title       text not null default 'Untitled board',
  visibility  board_visibility not null default 'link',
  share_token text unique,           -- random 22-char token for invite links
  link_role   member_role not null default 'editor', -- role granted via link
  thumbnail_url text,
  created_at  timestampz not null default now(),
  updated_at  timestampz not null default now()
);

create table board_members (
  board_id  uuid references boards(id) on delete cascade,
  user_id   uuid references profiles(id) on delete cascade,
  role      member_role not null default 'editor',
  joined_at timestampz not null default now(),
  primary key (board_id, user_id)
);

create table elements (
  id          uuid primary key,           -- generated on the client
```

```

board_id    uuid not null references boards(id) on delete cascade,
type        element_type not null,
data        jsonb not null check (pg_column_size(data) < 65536),
z_index     double precision not null default 0,
version     integer not null default 1,
created_by  uuid references profiles(id),
updated_by  uuid references profiles(id),
created_at  timestampz not null default now(),
updated_at  timestampz not null default now(),
deleted_at  timestampz                -- soft delete; enables restore later
);

-- indexes
create index on boards (owner_id, updated_at desc);
create index on board_members (user_id);
create index elements_live on elements (board_id, z_index) where deleted_at is null;
create index elements_sync on elements (board_id, updated_at);

```

Relationships

- A profile owns many boards; a board has many members through `board_members`.
- The owner is also stored in `board_members` with role `owner`, so every permission check reads one table.
- A board has many elements; deleting a board cascades to its members and elements.

Example data values

Type	data
pen	<code>{ "points": [[x,y,pressure], ...], "color": "#1e1e1e", "width": 3 }</code>
rect	<code>{ "x": 120, "y": 80, "w": 200, "h": 140, "stroke": "#1e1e1e", "fill": "transparent" }</code>
text	<code>{ "x": 40, "y": 60, "text": "Login flow", "fontSize": 20, "color": "#1e1e1e" }</code>

Row-level security. RLS is enabled on every table, and one helper function keeps the policies short.

```

create function board_role(b uuid) returns member_role
language sql stable security definer set search_path = public as $$
  select role from board_members where board_id = b and user_id = auth.uid()
$$;

-- members can read a board's elements
create policy "members read elements" on elements for select
  using (board_role(board_id) is not null);

-- only owners can rename or delete a board
create policy "owner updates board" on boards for update
  using (board_role(id) = 'owner');

```

Element writes go only through `apply_element_ops`, so there are no direct insert or update policies on `elements`.

Database functions (replaces the REST API)

With no Express server, all reads use `supabase-js` queries protected by RLS, and every write that needs logic runs as a Postgres function called with `supabase.rpc()`.

Action	Call	Allowed role
List my boards	<code>from('board_members').select('role, boards(*)')</code>	Signed-in user or guest
Load a board's elements	<code>from('elements').select().eq('board_id', id).is('deleted_at', null)</code>	Member
Create a board	<code>rpc('create_board', { title })</code> → board + owner membership	Signed-in user or guest
Rename or change link role	<code>from('boards').update(...)</code>	Owner
Delete a board	<code>from('boards').delete()</code>	Owner
Create or rotate invite link	<code>rpc('rotate_share_token', { board_id })</code>	Owner
Join through invite link	<code>rpc('join_board', { token })</code> → board id + role	Signed-in user or guest
Save element changes	<code>rpc('apply_element_ops', { board_id, ops })</code> → accepted + rejected	Editor or owner
List members	<code>from('board_members').select('role, profiles(display_name, avatar_url)')</code>	Member
Change or remove a member	<code>rpc('set_member_role', ...)</code> , <code>rpc('remove_member', ...)</code>	Owner, or self to leave
Upload thumbnail	<code>storage.from('thumbnails').upload(...)</code>	Editor or owner

Conventions. Functions raise errors with clear codes (`FORBIDDEN` , `NOT_FOUND` , `VERSION_CONFLICT`) that the client maps to toasts. Functions that change permissions run as `security definer` and check `auth.uid()` themselves.

Frontend structure

The frontend has five routes and keeps canvas drawing outside React's render cycle, so 60 fps drawing never waits on component updates.

Pages

Route	Page	Contents
<code>/</code>	Landing	Hero, demo GIF, "Start a board" (instant guest) and sign-in buttons
<code>/boards</code>	Dashboard	Board grid with thumbnails; banner asking guests to sign in to keep boards
<code>/b/:id</code>	Board	Canvas, toolbar, style panel, presence avatars, share dialog
<code>/join/:token</code>	Join	Asks for a name if new, joins, then redirects to the board
<code>/auth/callback</code>	OAuth callback	Finishes Google/GitHub sign-in or identity linking

Folder layout

```

src/
  pages/           Landing, Dashboard, Board, Join, AuthCallback
  components/      Toolbar, StylePanel, PresenceBar, ShareDialog, BoardCard, NameDialog
  canvas/
    renderer.ts    draws elements to the static and active layers
    tools/         pen.ts, shape.ts, text.ts, select.ts, eraser.ts
    hitTest.ts     finds the element under the pointer
    viewport.ts    pan/zoom math (screen ↔ world coordinates)
  realtime/
    channel.ts     join board channel, broadcast, presence
    sync.ts        optimistic apply, join buffering, rollback
    persistence.ts batches ops and calls apply_element_ops
  store/           boardStore.ts, uiStore.ts (Zustand)
  lib/             supabase.ts, auth.ts, schemas.ts (Zod)
  supabase/
    migrations/    tables, RLS policies, functions, triggers
    seed.sql       demo board for local development
    config.toml    local Supabase settings

```

Canvas rendering

- Two stacked canvases: a static layer with all committed elements, redrawn only when they change, and an active layer for the stroke being drawn, other users' in-progress strokes and cursors.
- Rendering runs in a `requestAnimationFrame` loop that redraws only when a dirty flag is set.
- Elements are stored in world coordinates; the viewport converts them with one transform, so pan and zoom never modify data.
- Pen strokes are smoothed with quadratic curves, and point lists are simplified before saving to cut payload size.
- Elements outside the visible viewport are skipped during drawing.

State

- `boardStore` : elements map, z-order, my undo/redo stacks, unsaved op queue, other users' presence.
- `uiStore` : active tool, stroke color and width, zoom level, open dialogs.
- Canvas code subscribes to the store directly instead of through React hooks.

Security, performance and deployment

The whole app runs on two free tiers: Vercel for the frontend and Supabase for everything else.

Security

- RLS on every table; element writes only through `apply_element_ops`, which checks the caller's role.
- Private Realtime channels, so non-members cannot listen to or send events on a board.
- Database CHECK constraints cap element `data` at 64 KB and display names at 40 characters; Zod validates the same rules on the client.
- Clients throttle broadcasts (cursors 1 per 50 ms, stroke points 1 per 30 ms); RPC batches every 500 ms.
- Text is rendered on the canvas, never as HTML, which removes XSS risk from board content.
- Only the Supabase anon key ships to the browser; the service-role key is never used in the frontend.
- Optional: CAPTCHA (Cloudflare Turnstile) on guest sign-up to stop bots from creating accounts.

Performance targets

Metric	Target
Stroke visible to other users	< 100 ms on the same region
Canvas frame rate while drawing	60 fps with 2,000 elements
Board load (elements query)	< 500 ms for 2,000 elements
Concurrent users per board	20

Deployment

Part	Host	Notes
Frontend	Vercel	Auto-deploy from <code>main</code> ; env: Supabase URL and anon key
Auth, Postgres, Realtime, Storage	Supabase	Migrations applied with <code>supabase db push</code>
CI	GitHub Actions	Lint, type-check and tests on every pull request; push migrations on <code>main</code>

Free-tier notes. Check Supabase's current pricing page for Realtime connection and message limits. Free Supabase projects may pause after a period of inactivity, so open or ping the demo regularly so it's awake when a recruiter visits.

Milestones

1. **Foundation:** Vite app, Supabase project, migrations, guest + Google + GitHub auth, dashboard CRUD.
2. **Solo canvas:** pen, shapes, text, select, pan/zoom, undo/redo, local only.
3. **Real-time:** board channel, broadcast events, live cursors, presence.
4. **Persistence:** `apply_element_ops` , version checks, join buffering, reconnect handling.
5. **Sharing:** invite links, roles, viewer mode, PNG export, thumbnails, guest upgrade.
6. **Ship:** deploy, two-browser Playwright test, README with demo GIF and architecture diagram.